

计算理论

第三部分 计算复杂性

第7章 时间复杂性

1. 时间复杂性

$\{ 0^k 1^k \mid k \geq 0 \}$ 的时间复杂性分析

2. 不同模型的运行时间比较

单带与多带 确定与非确定

3. P类与NP类

4. NP完全性及NP完全问题

二.

不同模型的时间复杂度比较

- 单带与多带
- 确定与非确定

单带与多带运行时间比较(P156-7)

$\{ 0^k 1^k \mid k \geq 0 \}$ 有 $O(n)$ 时间双带图灵机

M_3 = “对输入串 w :

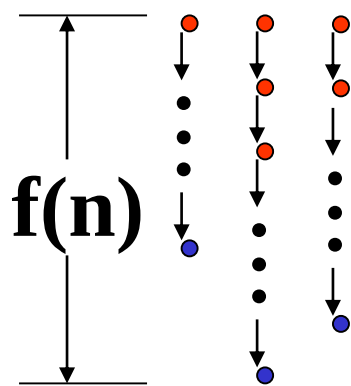
- 1) 扫描1带,如果在1的右边发现0,则拒绝.
- 2) 将1带的1复制到2带上.
- 3) 每删除一个1带的0就删除一个2带的1.
- 4) 如果两带上同时没有0和1,就接受.”

定理: 设函数 $t(n) \geq n$, 则每个 $t(n)$ 时间多带TM
和某个 $O(t^2(n))$ 时间单带TM等价.

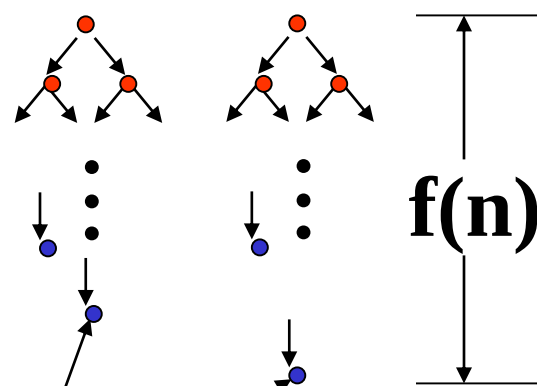
非确定判定器的运行时间(P157)

定义: 对非确定型判定器 N , 其运行时间 $f(n)$ 是在**所有**长为 n 的输入上, **所有**分支的最大步数.

时间 $f(n)$ 确定图灵机



时间 $f(n)$ 非确定图灵机

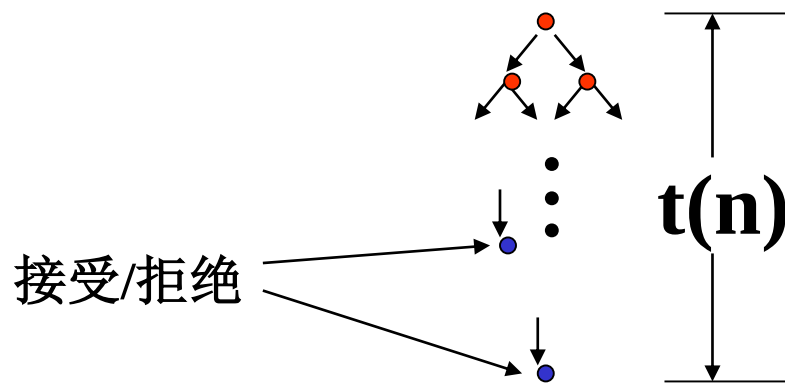


接受 或 拒绝

NTM的运行时间(P158)

定义: 对非确定型判定器 N , 其运行时间 $f(n)$ 是在所有长为 n 的输入上, 所有分支的最大步数.

定理: 设 $t(n) \geq n$, 则每个 $t(n)$ 时间NTM 都有一个 $2^{O(t(n))}$ 时间单带确定TM与之等价.



定理: 设 $t(n) \geq n$, 则 $\text{NTIME}(t(n)) \subseteq \text{TIME}(2^{O(t(n))})$

- ✓ $\text{TIME}(2^{O(t(n))})$: 确定型图灵机, 最坏运行时间不超过 $O(2^{O(t(n))})$ 的判定问题集合
- ✓ $\text{NTIME}(t(n))$: 非确定型图灵机, 在时间 $t(n)$ 内能解决的判定问题集合

计算理论

第三部分 计算复杂性

第7章 时间复杂性

1. 时间复杂性

$\{ 0^k 1^k \mid k \geq 0 \}$ 的时间复杂性分析

2. 不同模型的运行时间比较

单带与多带 确定与非确定

3. P类与NP类

4. NP完全性及NP完全问题

三. P与NP

多项式时间(P158)

运行时间相差多项式可以认为是小的
相差指数可以认为是大的.

例如: n^3 与 2^n ,对于 $n=1000$.

有关素性测试: $\text{Prime} = \{ p \mid p \text{是素数} \}$

如何编码? 一进制,二进制,十进制?

典型的指数时间算法来源于蛮力搜索.

有时通过深入理解问题可以避免蛮搜.

2001年Prime被证明存在多项式时间算法.

P类(P159)

定义:P是单带确定TM在
多项式时间内可判定的问题,即

$$P = \cup_k \text{TIME}(n^k)$$

P类的重要性在于:

- 1) 对于所有与单带确定TM等价的模型,P不变.
- 2) P大致对应于在计算机上实际可解的问题.
研究的核心是一个问题是否属于P类.

NP类(P165)

定义:NP是单带非确定TM在
多项式时间内可判定的问题,即

$$\text{NP} = \cup_k \text{NTIME}(n^k)$$

$$\text{EXP} = \cup_k \text{TIME}(2^{O(n^k)})$$

$$\text{P} \subseteq \text{NP} \subseteq \text{EXP}$$

$$\text{P} \subset \text{EXP}$$

NP类(P165)

$P \subseteq NP$ （所有 **P** 类问题，都是 **NP** 类问题）

- (1) 任何能用确定性图灵机在多项式时间解决的问题，也能用非确定性图灵机在多项式时间解决
- (2) 非确定性图灵机是更“强大”的模型，它包含了确定性图灵机的所有能力

$NP \subseteq EXP$

- (1) 任何非确定性多项式时间（**NP**）问题，都可以被确定性图灵机用暴力枚举的方式在指数时间内解决。
- (2) 用确定性图灵机穷举所有可能的非确定性分支，验证是否存在一个接受路径，这个过程的时间复杂度就是 $2^{O(n^k)}$ ，属于 **EXP** 类。

$P \subset EXP$ （**P** 是 **EXP** 的真子集）

- (1) 存在可以用指数时间解决，但无法用多项式时间解决的问题。
换句话说，**EXP** 中包含了比 **P** 更难的问题，且这些问题是确定存在的。
- (2) 指数时间算法的能力，确实比多项式时间算法更强，
它们能解决多项式时间算法无法处理的问题。

一些P问题(P159)

有些问题初看起来不属于P

求最大公因子: 欧几里德算法,

辗转相除法

模p指数运算 $a^b \bmod p$

上下文无关语言 有 $O(n^3)$ 判定器

素性测试 等等

以增加空间复杂性来减小时间复杂性

快速验证(P163)

HP = {<G,s,t> | G是包含从s到t的
哈密顿路径的有向图}

CLIQUE = {<G,k> | G是有k团的无向图}

目前没有快速算法,但其成员是可以快速验证的.

注意:**HP**的补可能不是可以快速验证的.

快速验证的特点:

1. 只需要对语言中的串能快速验证.
2. 验证需要借助额外的信息:证书,身份证.

快速验证

- 目前没有快速算法，但其成员是可以快速验证的
 - 对于 HP/CLIQUE，目前没有多项式时间的确定性算法来直接“求解”
 - 但如果有人给你一个“答案”（比如一条路径，或者一个顶点集合）(证书)，你可以用多项式时间验证它是不是正确的

NP问题(P165)

团:无向图的完全子图(所有节点都有边相连).

CLIQUE = { $\langle G, k \rangle$ | G 是有 k 团的无向图 }

定理: **CLIQUE** $\in$ NP.

N=“对于输入 $\langle G, k \rangle$,这里 G 是一个图:

- 1)非确定地选择 G 中 k 个节点的子集 c .
- 2)检查 G 是否包含连接 c 中节点的所有边.
- 3)若是,则接受;否则,拒绝.”

哈密顿路径问题 $HP \in NP$ (对比P164)

HP = { $\langle G, s, t \rangle$ | G 是包含从 s 到 t 的
哈密顿路径的有向图 }

P 时间内判定 **HP** 的 **NTM**:

N_1 = “对于输入 $\langle G, s, t \rangle$:

- 1) 非确定地选 G 的所有节点的排列 $p_1, \dots, p_m$.
- 2) 若 $s = p_1, t = p_m$, 且对每个 i , (p_i, p_{i+1}) 是 G 的边, 则接受; 否则拒绝.”

P与NP(P166)

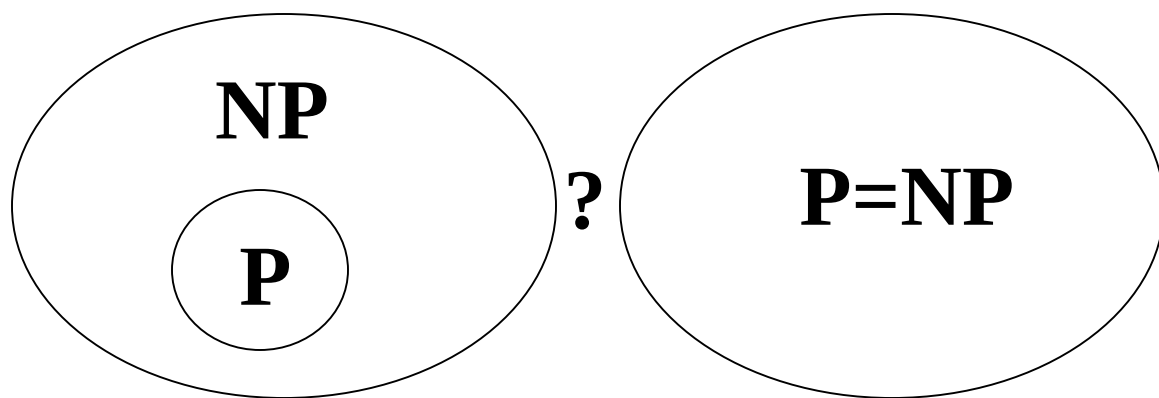
P=成员资格可以**快速判定**的语言类.

NP=成员资格可以**快速验证**的语言类.

显然有 $P \subseteq NP$

但是否有 $P = NP$?

看起来难以想象,但是现在没有证明.



当代数学与
理论计算机
共同的难题.

P与NP

- **P**=成员资格可以**快速判定**的语言类。
 - **含义**：存在确定性算法，能直接在多项式时间内给出“是 / 否”的最终结果。
 - **举例**：给一串数字判断是否升序排列，我们顺着遍历一次就能直接得出结论，属于 **P** 问题。
 - **特点**：从无到有直接算出答案，全程高效。
- **NP**=成员资格可以**快速验证**的语言类。
 - **含义**：不一定能快速直接求解，但如果有人给了一份证书（候选答案），我们可以在多项式时间内核验这份答案对不对。
 - **举例**：哈密顿路径问题，很难直接快速找出路径，但别人给出一条路径后，我们能快速检查它是否真的满足哈密顿路径的要求。
 - **特点**：验证容易、直接求解很难

计算理论

第三部分 计算复杂性

第7章 时间复杂性

1. 时间复杂性

$\{ 0^k 1^k \mid k \geq 0 \}$ 的时间复杂性分析

2. 不同模型的运行时间比较

单带与多带 确定与非确定

3. P类与NP类

4. NP完全性及NP完全问题

四. NP完全

- NP完全性的定义
 - SAT是NP完全问题
 - 一些NP完全问题
-
- SAT: 布尔公式可满足性问题

NP完全性(P166)

Cook(美)和Levin(苏联)于1970's证明

NP中某些问题的复杂性与

整个NP类的复杂性相关联, 即:

若这些问题中的任一个找到P时间算法, 则 $P=NP$.

这些问题称为NP完全问题.

理论意义: 两方面

- 1) 研究P与NP关系可以只关注于一个问题的算法.
- 2) 可由此说明一个问题目前还没有快速算法.

合取范式(P167-8)

- **布尔变量**: 取值为1和0(True, False)的变量.
- **布尔运算**: AND($\wedge$),OR ($\vee$),NOT ($\neg$). **布尔公式**.
例: $\phi_1 = ((\neg x) \wedge y) \vee (x \wedge (\neg z))$, $\phi_2 = (\neg x) \wedge x$
- 称 ϕ **可满足**, 若存在布尔变量的0,1赋值使得 $\phi=1$. 例 ϕ_1, ϕ_2 .
 ϕ **不可满足** $\Leftrightarrow \neg \phi$ 永真
- **文字**: 变量或变量的非,如 x 或 $\neg x$.
- **子句**: 由 $\vee$ 连接的若干文字,如 $x_1 \vee (\neg x_2) \vee x_3 \vee x_4$.
- **合取范式(cnf)**: 由 $\wedge$ 连接的若干子句,如 $((\neg x_1) \vee x_2 \vee (\neg x_3)) \wedge (x_2 \vee (\neg x_3) \vee x_4 \vee x_5) \wedge ((\neg x_4) \vee x_5)$
- **k-cnf** (conjunctive normal form)
每个子句的文字数不大于k: 3cnf, 2cnf

可满足问题SAT(P167-8)

- 可满足性问题:

$SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的布尔公式} \}$ **NP完全**

- 二元可满足性问题:

$2SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的2cnf} \} \in P$

- 三元可满足性问题:

$3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的3cnf} \}$ **NP完全**

2SAT问题: 给定一个由合取范式 (CNF) 表示的布尔公式, 其中每个子句最多包含2 个文字, 判断是否存在一组布尔变量的赋值, 使得整个公式的值为真。

可满足问题SAT

- $SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的布尔公式} \}$
 - SAT (Satisfiability Problem)是一个语言，也就是所有“可满足布尔公式的编码”组成的集合
 - $\langle \phi \rangle$ 表示公式 ϕ 的编码
 - SAT 包含所有这样的输入编码 $\langle \phi \rangle$ ，其中 ϕ 是可满足的布尔公式

二元可满足问题 2SAT $\in P$ (ex7.23)

1. 当2cnf中有子句是单文字 x , 则反复执行(直接)清洗

1.1 由 x 赋值, 1.2 删去含 x 的子句, 1.3 删去含 $\neg x$ 的文字
若清洗过程出现相反单文子子句, 则清洗失败并结束

$$\begin{aligned} & (x_1 \vee x_2) \wedge (x_3 \vee \neg x_2) \wedge (x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4) \\ & \rightarrow (x_3 \vee \neg x_2) \wedge (\neg x_2) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4) \\ & \rightarrow (x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4) \end{aligned}$$

2. 若无单文字子句, 则任选变量赋真/假值各(赋值)清洗一次
若两次都清洗失败, 则回答不可满足.

$$x_3=1 \rightarrow (x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (x_4) \rightarrow (\neg x_4) \wedge (x_4) \text{ 失败}$$

$$x_3=0 \rightarrow (x_4) \wedge (\neg x_4 \vee \neg x_5) \rightarrow (\neg x_5) \rightarrow \emptyset \text{ 成功}$$

3. 若成功清洗后有子句剩下, 则继续2. 否则, 回答可满足.

注: 见[S]习题7.23, 作者答案与清洗算法等价. 贪心.

二元可满足问题

$$(x_1 \vee x_2) \wedge (x_3 \vee \neg x_2) \wedge (x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4)$$

1. 发现单文字子句 (x_1) , 所以 $x_1 = 1$:

- 删掉含 x_1 的子句: $(x_1 \vee x_2)$ 和 (x_1) 被删除。
- 删掉含 $\neg x_1$ 的文字: $(\neg x_1 \vee \neg x_2)$ 中 $\neg x_1$ 被删, 剩下 $(\neg x_2)$, 成为新的单文字子句。

公式变为:

$$(x_3 \vee \neg x_2) \wedge (\neg x_2) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4)$$

2. 现在有单文字子句 $(\neg x_2)$, 所以 $x_2 = 0$:

- 删掉含 $\neg x_2$ 的子句: $(x_3 \vee \neg x_2)$ 和 $(\neg x_2)$ 被删除。

公式变为:

$$(x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4)$$

二元可满足问题

$$(x_3 \vee x_4) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4)$$

我们选变量 x_3 来尝试：

第一次尝试： $x_3 = 1$

- 删掉含 x_3 的子句： $(x_3 \vee x_4)$ 和 $(\neg x_3 \vee x_5)$ 、 $(\neg x_3 \vee x_4)$ 被删除。
- 删掉含 $\neg x_3$ 的文字：公式里没有 $\neg x_3$ ，所以剩下：

$$(x_5) \wedge (x_4) \wedge (\neg x_4 \vee \neg x_5)$$

现在有单文字子句 (x_5) 和 (x_4) ：

- $x_5 = 1 \rightarrow$ 删掉含 x_5 的子句 $(\neg x_4 \vee \neg x_5)$ ，但同时出现 $(\neg x_4)$ 。
- $x_4 = 1$ 和 $(\neg x_4)$ 矛盾 $\rightarrow$ 清洗失败。

第二次尝试： $x_3 = 0$

- 删掉含 $\neg x_3$ 的子句： $(\neg x_3 \vee x_5)$ 和 $(\neg x_3 \vee x_4)$ 被删除。
- 删掉含 x_3 的文字： $(x_3 \vee x_4)$ 中 x_3 被删，剩下 (x_4) ，成为单文字子句。

公式变为：

$$(x_4) \wedge (\neg x_4 \vee \neg x_5)$$

继续清洗：

- $x_4 = 1 \rightarrow$ 删掉含 x_4 的子句 (x_4) ，并删掉 $(\neg x_4 \vee \neg x_5)$ 中的 $\neg x_4$ ，剩下 $(\neg x_5)$ 。
- $x_5 = 0$ ，无矛盾，清洗成功，公式变为空。

3SAT $\in$ NP(P173)

三元可满足性问题:

$$3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的 3cnf} \}$$

P时间内判定3SAT的NTM:

N=“对于输入 $\langle \phi \rangle$, ϕ 是一个3cnf公式,

1)非确定地选择各变量的赋值T.

2)若在赋值T下 $\phi=1$, 则接受;否则拒绝.”

第2步在公式长度的多项式时间内运行.

3SAT $\in$ NP

- 给定一个 3CNF 形式的布尔公式，判断是否存在变量赋值使公式为真，这个判定问题就是 3SAT。

3SAT ∈ P? (补充)

$3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的 3cnf} \}$

清洗算法对 3cnf 是否有效? 举例对比:

$(x_3 \vee \neg x_3) \wedge (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$

$x_3=1$ 清洗无矛盾; $x_1=0$ 和 1 都清洗失败, **不可满足**.

$(x_3 \vee \neg x_3) \wedge (\neg x_3 \vee x_1 \vee x_2) \wedge (\neg x_3 \vee x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_1 \vee x_2) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2)$

$x_3=1$ 清洗无矛盾; $x_1=0$ 和 1 都清洗失败, **返 $x_3=0$**

3cnf 清洗不能避免搜索, 指数时间.

目前还不知道 3SAT 是否属于 P.

归约引理: 若 $A \leq_p B$ 且 $B \in P$, 则 $A \in P$ (P168)

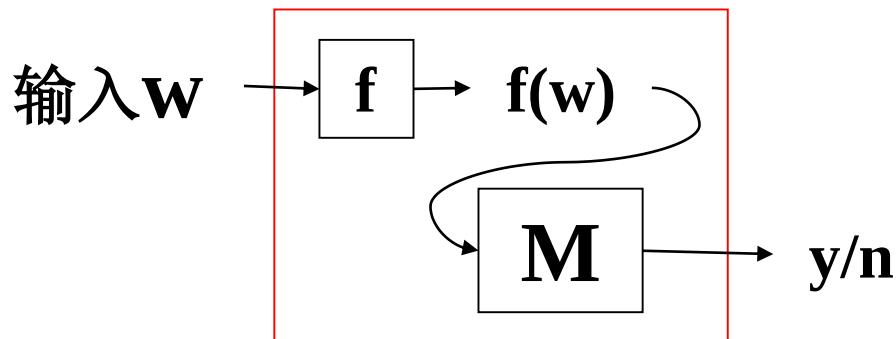
- 定义: 多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$. (例如 $f(u) = u0$)
若 $\exists$ 多项式时间图灵机, $\forall w$ 输入, 停机时带上的串为 $f(w)$
- 定义: 称 A 可多项式时间映射归约到 B ($A \leq_p B$),

若存在多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$,

$$\forall w \in \Sigma^*, w \in A \Leftrightarrow f(w) \in B.$$

函数 f 称为 A 到 B 的多项式时间归约.

通俗地说: f 将 A 的实例编码转换为 B 的实例编码.



利用 f 和 B 的判定器
构造 A 的判定器

C-L定理: $SAT \in P \Leftrightarrow P = NP$ (P167-8)

- **定义:** 多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$. (例如 $f(u) = u0$)
若 $\exists$ 多项式时间图灵机, $\forall w$ 输入, 停机时带上的串为 $f(w)$
- **定义:** 称 A 可多项式时间映射归约到 B ($A \leq_p B$),
若存在多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$,
$$\forall w \in \Sigma^*, w \in A \Leftrightarrow f(w) \in B.$$

函数 f 称为 A 到 B 的多项式时间归约.
通俗地说: f 将 A 的实例编码转换为 B 的实例编码.
- **Cook-Levin定理:** 对任意 $A \in NP$ 都有 $A \leq_p SAT$.
- **归约引理:** 若 $A \leq_p B$, 且 $B \in P$, 则 $A \in P$.
- **推论:** 若 $SAT \in P$, 则 $NP = P$.

推论：若 $SAT \in P$, 则 $NP = P$.

证明 “若 $SAT \in P$, 则 $P=NP$ ”

任取一个 $A \in NP$ 。由 Cook-Levin 定理可知：

$$A \leq_p SAT$$

如果再假设：

$$SAT \in P$$

那么由归约引理得到：

$$A \in P$$

公式解释：因为 A 是任意选取的 NP 问题，所以每一个 NP 问题都属于 P 。这可以写作：

$$NP \subseteq P$$

又因为本来就有：

$$P \subseteq NP$$

所以得到：

$$P = NP$$

归约定理: 若 $A \leq_p B$ 且 $B \in P$, 则 $A \in P$ (P168)

证明: 设 $f: \Sigma^* \rightarrow \Sigma^*$ 是 A 到 B 的 P 时间归约,

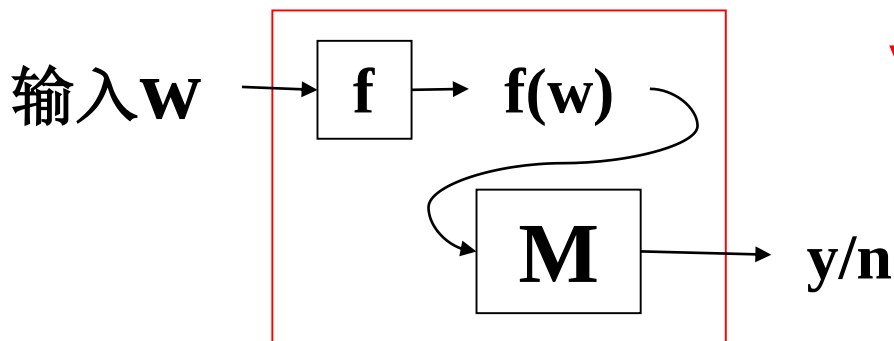
B 有 P 时间判定器 M , 则

$N =$ “输入 w , 计算 $M(f(w))$, 输出 M 的运行结果”

在多项式时间内判定 A .

问题: 若 f 是 n^a 时间归约, M 是 n^b 时间判定器, 则 N 时间?

设 $|w| = n$, 则 $|f(w)| \leq n^a$, 则 $M(f(w))$ 时间 $\leq n^{ab}$. (多项式时间)



$$\forall w \in \Sigma^*, w \in A \Leftrightarrow f(w) \in B.$$

利用 f 和 B 的判定器
构造 A 的判定器

设 $|w|=n$, 则 $|f(w)| \leq n^a$, 则 $M(f(w))$ 时间 $\leq n^{ab}$.

设输入长度为:

$$|w| = n$$

公式解释: $|w|$ 表示输入串 w 的长度, 记为 n 。复杂性分析都要以输入长度为变量。

如果 f 在 $O(n^a)$ 时间内计算完成, 那么它输出的字符串长度不会超过它运行过的时间量级。因此可以写成:

$$|f(w)| \leq O(n^a)$$

公式解释: 输出 $f(w)$ 本身也要被写在带上。如果机器只运行 $O(n^a)$ 步, 不可能写出超过 $O(n^a)$ 长度的输出。因此 $f(w)$ 的长度也是多项式规模。

再把 $f(w)$ 输入判定器 M 。 M 对长度为 m 的输入用时 $O(m^b)$, 这里 $m = |f(w)|$, 于是:

$$T_M(f(w)) \leq O(|f(w)|^b)$$

将 $|f(w)| \leq O(n^a)$ 代入, 得到:

$$T_M(f(w)) \leq O((n^a)^b) = O(n^{ab})$$

公式解释: $(n^a)^b = n^{ab}$, 这是幂的乘方法则。也就是说, 虽然先做了一次归约, 再运行一次 B 的算法, 但整体仍然只是多项式时间。

因此 N 的总时间为:

$$T_N(n) = O(n^a) + O(n^{ab})$$

公式解释: 第一项是计算 $f(w)$ 的时间, 第二项是运行 $M(f(w))$ 的时间。两个多项式相加仍然是多项式, 所以 N 是多项式时间判定器。

定理: $3SAT \leq_p CLIQUE$ (P168)

$3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的 3cnf 公式} \}$

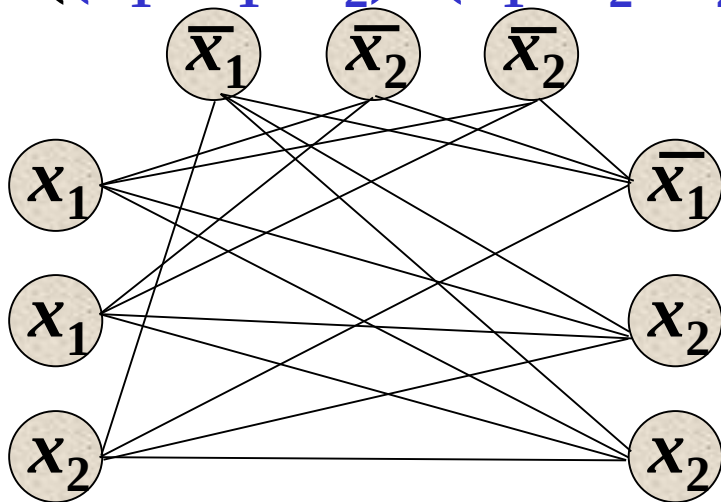
$CLIQUE = \{ \langle G, k \rangle \mid G \text{ 是有 } k \text{ 团的无向图} \}$.

证明: 设 $\phi = (a_1 \vee b_1 \vee c_1) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$, 有 k 个子句.

令 $f(\phi) = \langle G, k \rangle$, G 有 k 组节点, 每组 3 个;

同组节点无边相连, 相反标记无边相连.

例: $f((x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)) = \langle G, 3 \rangle$



需证: $\langle \phi \rangle \in 3SAT$

$\Leftrightarrow$

$\langle (G, k) \rangle \in CLIQUE$

定理: $3SAT \leq_p CLIQUE$ (P168)

- $CLIQUE = \{ \langle G, k \rangle \mid G \text{ 是有 } k \text{ 团的无向图} \}$.
 - 表示图中有 k 个顶点两两之间都有边相连。

$\forall \phi, \phi \in 3SAT \Leftrightarrow f(\phi) \in CLIQUE$ (P169)

$\langle \phi \rangle (\langle (x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2) \rangle) \in 3SAT$

$\Leftrightarrow \exists$ 变量赋值 $(x_1=0, x_2=1)$ 使得 $\phi=1$

$\Leftrightarrow \exists k$ 团 (每组挑一个真顶点得到 k 团, 非同组非相反)

$\Leftrightarrow f(\phi) (\langle G, 3 \rangle) \in CLIQUE.$

F 在 $|\langle \phi \rangle|$ 的多项式时间内可计算:

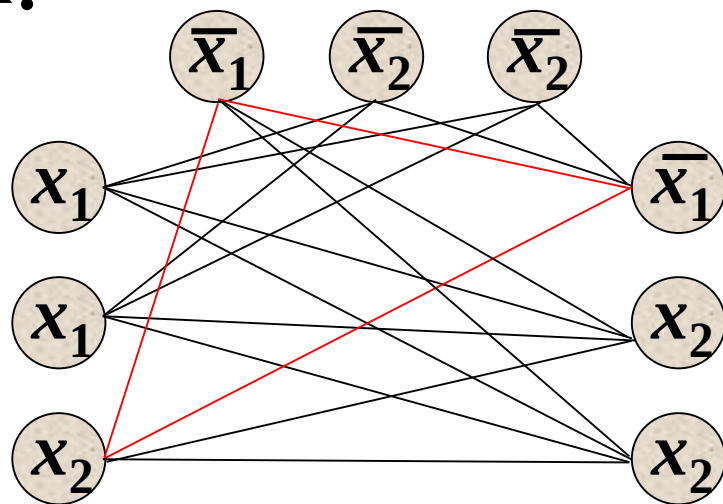
设 ϕ 的长度 $3k = O(k)$,

则 G 的顶点数 $3k = O(k)$,

G 的边数是 $O(k^2)$

可见 $f(\phi) = \langle G, k \rangle$ 的构造

可在 k 的多项式时间内完成.



步骤 1：给出 3SAT 实例

$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$

这是一个 3CNF 公式，有 3 个子句，每个子句最多 3 个文字。

- 问题：是否存在变量赋值让 $\phi = 1$ ？

步骤 2：3SAT 可满足的等价表述

$\phi \in 3SAT \iff \exists \text{变量赋值}(x_1 = 0, x_2 = 1) \text{使得} \phi = 1$

- 验证：代入 $x_1 = 0, x_2 = 1$
 - 子句 1: $0 \vee 0 \vee 1 = 1$ ✓
 - 子句 2: $1 \vee 0 \vee 0 = 1$ ✓ ($\overline{x_1} = 1, \overline{x_2} = 0$)
 - 子句 3: $1 \vee 1 \vee 1 = 1$ ✓ ($\overline{x_1} = 1, x_2 = 1$)
- 所有子句都为真，因此 ϕ 是可满足的。

步骤 3：3SAT 到 CLIQUE 的图构造规则

这一步是核心，我们要把公式 ϕ 转换成一个图 G ，使得“ ϕ 可满足”等价于“ G 中存在大小为 k 的团”（这里 k 就是子句数，例子中 $k = 3$ ）。

构造规则：

- 分组**：把每个子句里的文字作为一组顶点。例子中 3 个子句对应 3 组顶点：
 - 组 1: x_1, x_1, x_2
 - 组 2: $\overline{x_1}, \overline{x_2}, \overline{x_2}$
 - 组 3: $\overline{x_1}, x_2, x_2$
- 连边规则**：两个顶点之间连边，当且仅当：
 - 它们**不在同一组**，且
 - 它们不是互补文字（比如 x_1 和 $\overline{x_1}$ 不能连边，因为赋值时不可能同时为真）。

步骤 4：可满足赋值 $\iff$ k 团的对应关系

- 赋值 $x_1 = 0, x_2 = 1$ 时，哪些文字为真？
 - 组 1: $x_2 = 1$ (选 x_2)
 - 组 2: $\overline{x_1} = 1$ (选 $\overline{x_1}$)
 - 组 3: $x_2 = 1$ (选 x_2)
- 这 3 个顶点: x_2 (组 1)、 $\overline{x_1}$ (组 2)、 x_2 (组 3)，它们两两之间都有边，构成了一个**大小为 3 的团**。
- 反过来，如果图中存在一个大小为 3 的团，那么团里的顶点一定来自不同的组，且没有互补文字，对应一个让所有子句为真的赋值。

步骤 5：最终结论

$\exists k\text{团} \iff f(\phi) = \langle G, 3 \rangle \in CLIQUE$

NP完全性(P169,175)

- 定义:语言B称为**NP完全**的(NPC),若它满足:

1) $B \in NP$; 2) $\forall A \in NP$, 都有 $A \leq_p B$.

- 定理1: $A \leq_p B + B \in P \Rightarrow A \in P$.

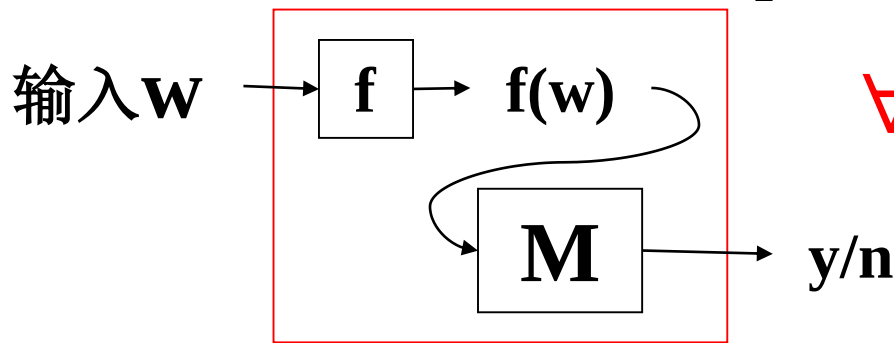
- 定理2: 若B是NPC, 且 $B \in P$, 则 $P=NP$.

证明: $\forall A \in NP$, $A \leq_p B + B \in P \Rightarrow A \in P$

- 定理3: 若B是NPC, $B \leq_p C$, 且 $C \in NP$, 则C是NPC.

证明: $\forall A \in NP$, $(A \leq_p B) + (B \leq_p C) \Rightarrow A \leq_p C$

- $3SAT$ 是NPC + $3SAT \leq_p CLIQUE \Rightarrow CLIQUE$ 是NPC



$\forall w \in \Sigma^*$, $w \in A \Leftrightarrow f(w) \in B$.

利用f和B的判定器
构造A的判定器

定理2: 若B是NPC, 且 $B \in P$, 则 $P=NP$.

$$B \in NPC \wedge B \in P \Rightarrow P = NP$$

公式解释: 如果一个 NP 完全问题 B 有多项式时间算法, 那么整个 NP 都会被拉进 P 。

证明过程如下。因为 B 是 NP 完全的, 所以:

$$\forall A \in NP, A \leq_p B$$

又因为:

$$B \in P$$

由定理 1 可得:

$$\forall A \in NP, A \in P$$

这说明:

$$NP \subseteq P$$

而本来就有:

$$P \subseteq NP$$

所以:

$$P = NP$$

直观理解: NP 完全问题像整个 NP 类的“总开关”。只要其中任意一个被证明可以多项式时间求解, 整个 NP 类就都可以多项式时间求解。

定理3: 若B是NPC, $B \leq_p C$, 且 $C \in NP$, 则C是NPC.

$$B \in NPC \wedge B \leq_p C \wedge C \in NP \Rightarrow C \in NPC$$

公式解释: 如果 B 已经是 NP 完全问题, 并且 B 可以多项式时间归约到 C , 同时 C 本身属于 NP , 那么 C 也是 NP 完全问题。

证明分两步。

第一步, 因为 $C \in NP$, 满足 NP 完全定义的第一个条件。

第二步, 要证明所有 $A \in NP$ 都能归约到 C 。由于 $B \in NPC$, 有:

$$\forall A \in NP, \quad A \leq_p B$$

又已知:

$$B \leq_p C$$

归约可以传递, 所以:

$$A \leq_p B \wedge B \leq_p C \Rightarrow A \leq_p C$$

公式解释: 如果有函数 f 把 A 转成 B , 又有函数 g 把 B 转成 C , 那么复合函数 $g(f(w))$ 就能把 A 转成 C 。两个多项式时间函数的复合仍然是多项式时间函数。

因此:

$$\forall A \in NP, \quad A \leq_p C$$

结合 $C \in NP$, 得到:

$$C \in NPC$$

Cook-Levin定理的证明步骤(补充)

- 定义:语言B称为NP完全的(NPC),若它满足:

1) $B \in NP$;

2) $\forall A \in NP$, 都有 $A \leq_p B$.

- Cook-Levin定理: SAT是NP完全问题.

证明步骤:

1. $SAT \in NP$ (已证)

2. $\forall A \in NP, A \leq_p SAT$

$\forall A \in \text{NP}$, 都有 $A \leq_p \text{SAT}$ (P170)

- 思想: 将字符串对应到布尔公式
利用接受的形式定义.
- 过程: 任取 $A \in \text{NP}$, 设 N 是 A 的 n^k 时间 NTM.
 $\forall w (|w|=n), N$ 接受 w
 - $\Leftrightarrow N$ 对 w 有长度小于 n^k 的接受格局序列
 - $\Leftrightarrow$ 能填好 N 在 w 上的画面 (一个 $n^k \times n^k$ 表格)
 - $\Leftrightarrow \phi = f(w)$ 可满足 ($|\langle \phi \rangle| = O(n^{2k})$)
- 结论: SAT 是 NP 完全的

$\forall A \in \text{NP}$, 都有 $A \leq_p \text{SAT}$

核心思想：用布尔公式描述 NTM 的计算过程

“将字符串对应到布尔公式，利用接受的形式定义”

- 本质：把一台非确定型图灵机 (NTM) 对输入 w 的“接受格局序列”，编码成一个布尔公式 ϕ_w 。

- 关键等价关系：

NTM 接受 $w \iff$ 公式 ϕ_w 是可满足的

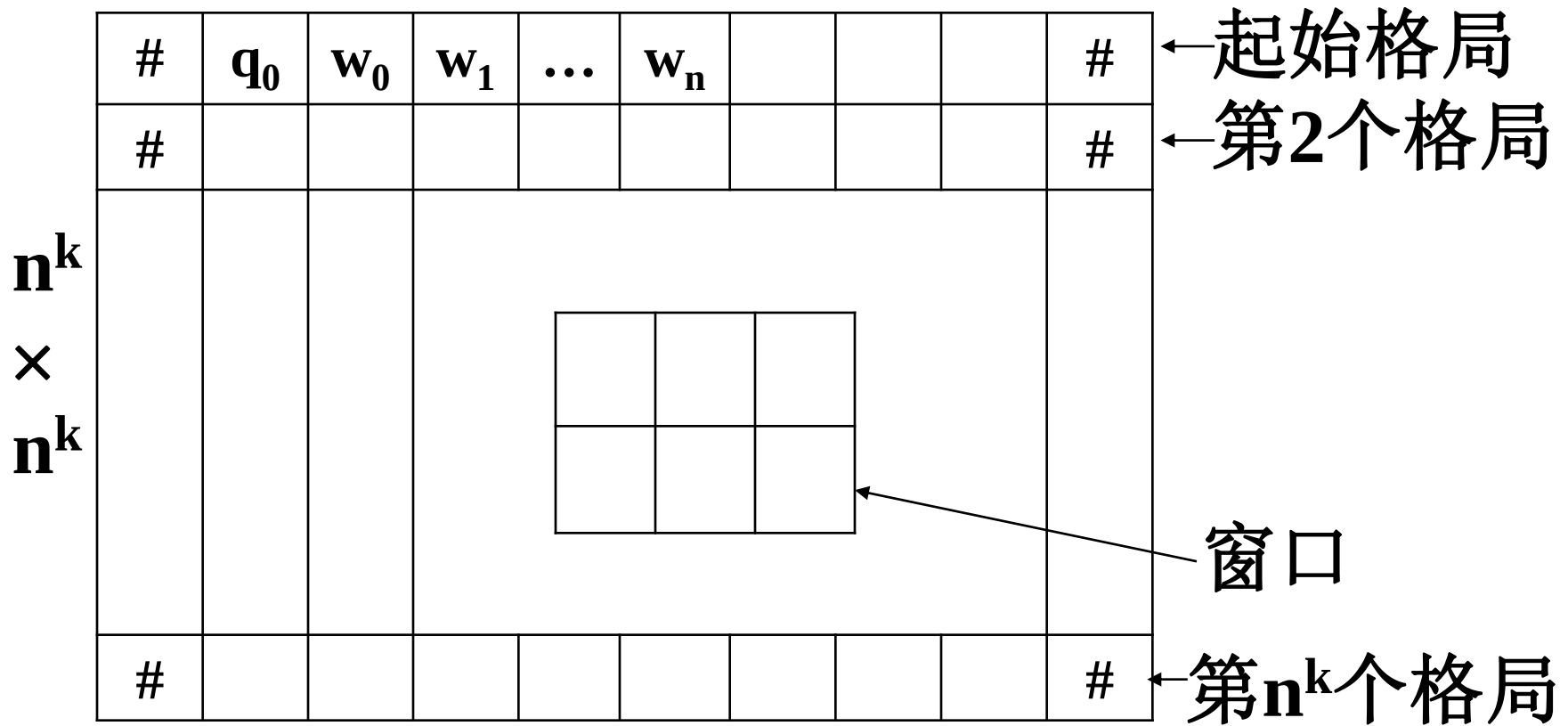
这样，判断 $w \in A$ ，就等价于判断 $\phi_w \in \text{SAT}$ ，

从而实现从A到SAT的归约。

结论：SAT 是 NP 完全的

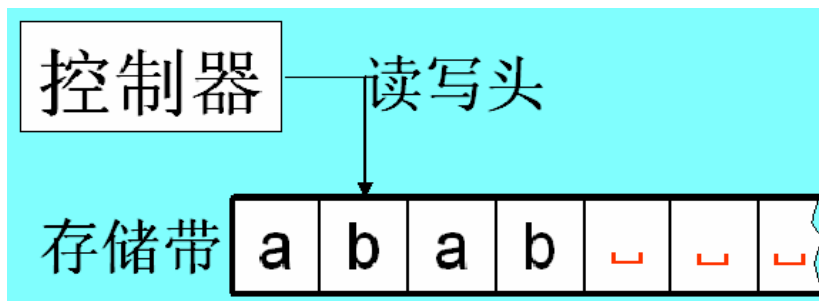
- 我们证明了：任意 NP 问题 A 都可以在多项式时间内规约到 SAT；
- 又因为之前已经证明了 $\text{SAT} \in \text{NP}$ ；
- 因此，SAT 满足 NP 完全问题的两个定义条件，是 NP 完全问题。

N 接受 $w \Leftrightarrow$ 能填好 N 在 w 上的表(P170)



能填好表: 第一行是起始格局
上一行能产生(或等于)下一行
表中有接受状态

回忆图灵机(TM)形式化定义(P88)

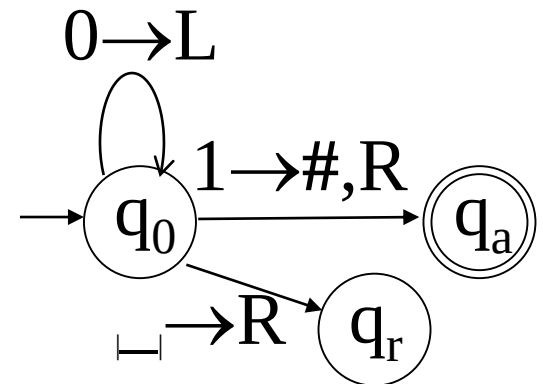


TM是一个7元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$

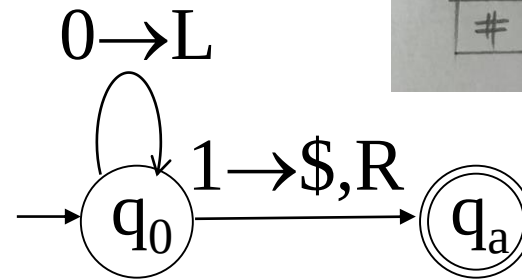
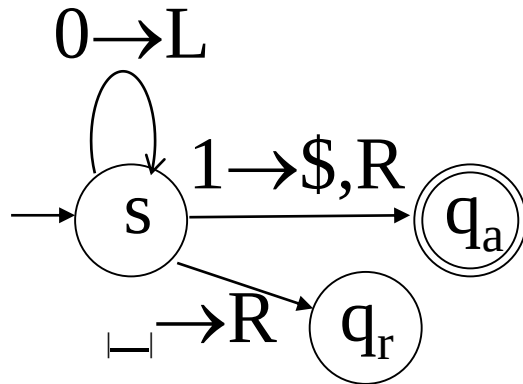
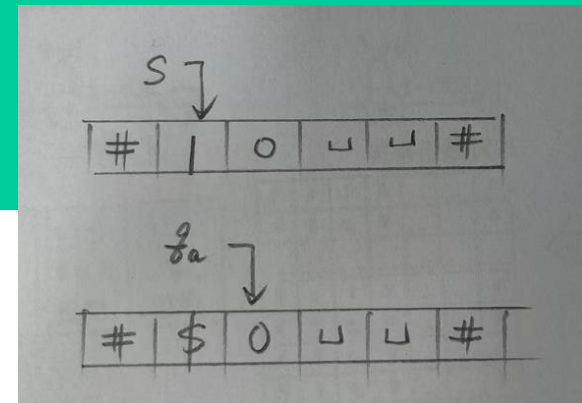
- 1) Q 是状态集.
- 2) Σ 是输入字母表,不包括空白符 $\sqcup$.
- 3) Γ 是带字母表,其中 $\sqcup \in \Gamma, \Sigma \subset \Gamma$.
- 4) $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 是转移函数.
- 5) $q_0 \in Q$ 是起始状态. 6) $q_a \in Q$ 是接受状态.
- 7) $q_r \in Q$ 是拒绝状态, $q_a \neq q_r$.

回忆图灵机格局的定义(P88-9)

- 描述图灵机运行的每一步需要如下信息：
控制器的**状态**；存储带上**字符串**；读写头的**位置**.
- 定义：对于图灵机 $M=(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ ，
设 $q \in Q$, $u, v \in \Gamma^*$ ，则**格局** uqv 表示
 - 1) 当前控制器**状态**为 q ；
 - 2) **存储带**上字符串为 uv (其余为空格)；
 - 3) **读写头**指向 v 的第一个符号.
- 起始格局, 接受格局, 拒绝格局.



格局演化举例(补充)



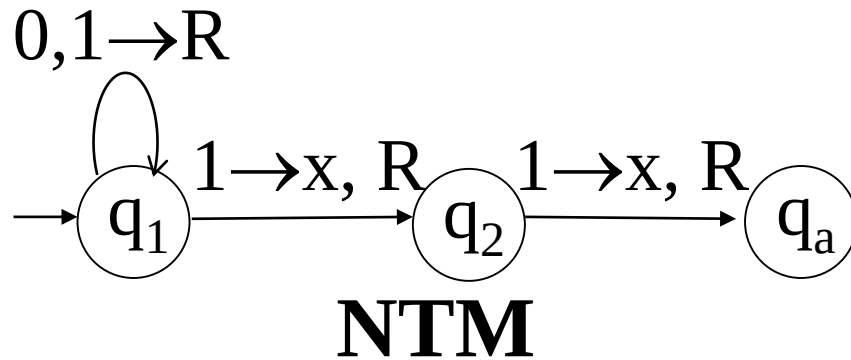
省略拒绝状态

s 0 1	s 1 0	s _ _
s 0 1	\$ q _a 0	_ q _r _
...	接受	拒绝
循环		

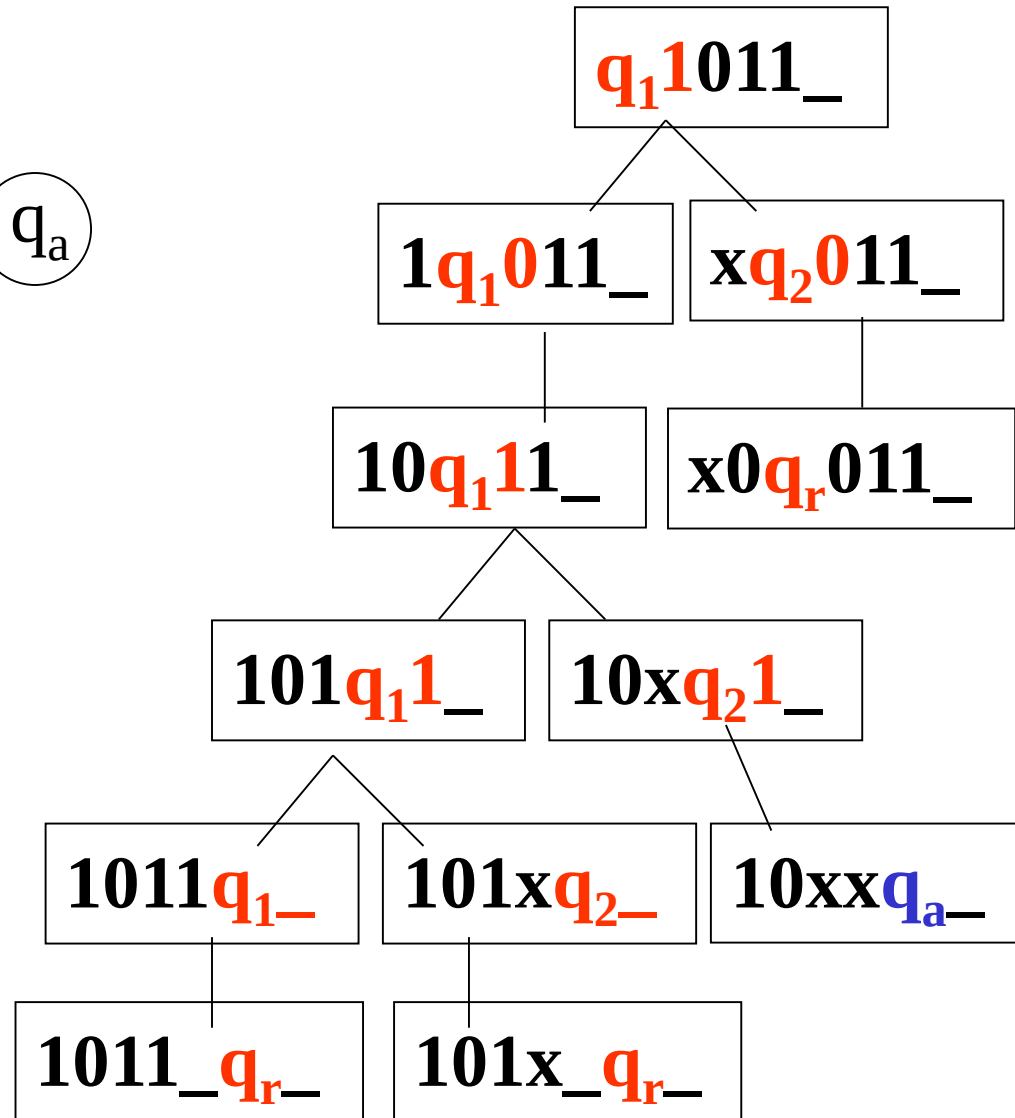
#	s	1	0	_	_	#
#	\$	q _a	0	_	_	#

#	s	1	0	_	_	#
#	\$	q _a	0	_	_	#
#	\$	q _a	0	_	_	#
#	\$	q _a	0	_	_	#

N接受 $w \Leftrightarrow$ 能填好N在 w 上的表(补充)



#	q_1	1	0	1	1	_	#
#	1	q_1	0	1	1	_	#
#	1	0	q_1	1	1	_	#
#	1	0	x	q_2	1	_	#
#	1	0	x	x	q_a	_	#
#	1	0	x	x	q_a	_	#
#	1	0	x	x	q_a	_	#
#	1	0	x	x	q_a	_	#



构造布尔公式 $\phi=f(w)$ (补充)

- 能填好画面 $\Leftrightarrow \phi=f(w)$ 可满足
- $f(w)=\langle \phi \rangle$, $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$ •
- 对于任意赋值:
 1. $\phi_{\text{cell}}=1 \Leftrightarrow$ 每格有且只有一个符号;
 2. $\phi_{\text{start}}=1 \Leftrightarrow$ 第一行是起始格局;
 3. $\phi_{\text{accept}}=1 \Leftrightarrow$ 表格中有接受状态;
 4. $\phi_{\text{move}}=1 \Leftrightarrow$ 每行由上一行格局产生.
- $\forall w, w \in A \Leftrightarrow \langle \phi \rangle \in \text{SAT}$ 即 $A \leq_m \text{SAT}$
- 若 $|\langle \phi \rangle|$ 是 $|w|$ 的多项式, 则有 $A \leq_p \text{SAT}$

构造布尔公式 $\phi=f(w)$

- $\forall w, w \in A \Leftrightarrow \langle \phi \rangle \in SAT$ 即 $A \leq_m SAT$

若 ϕ 可满足，则存在一组布尔变量赋值，使得四个条件都成立。于是我们可以根据这些变量的真假值读出一张表。由于 ϕ_{cell} 保证每格唯一， ϕ_{start} 保证第一行正确， ϕ_{move} 保证每一步合法， ϕ_{accept} 保证出现接受状态，所以这张表就是一张合法接受表。

反过来，如果 N 在 w 上有一张合法接受表，那么让对应每个格子真实符号的变量取 1，其余变量取 0，就能使四个子公式都为真。因此 ϕ 可满足。

这个关系可以写成：

$$w \in A \Leftrightarrow \langle \phi \rangle \in SAT.$$

公式解释： $w \in A$ 表示原问题的输入是“是实例”； $\langle \phi \rangle \in SAT$ 表示构造出的布尔公式可满足。二者等价，说明 f 的确是从 A 到 SAT 的归约。

构造布尔公式 $\phi=f(w)$

- 若 $|\langle\phi\rangle|$ 是 $|w|$ 的多项式, 则有 $A \leq_p SAT$

$$\forall w, \quad w \in A \Leftrightarrow \langle\phi\rangle \in SAT.$$

公式解释：这不是只对某个特殊输入成立，而是对任意输入 w 都成立。所以 f 是一个正确的映射归约。

因此可以得到：

$$A \leq_m SAT.$$

公式解释： $\leq_m$ 表示映射归约。这里强调的是把 A 的每个实例 w 映射为 SAT 的实例 $\langle\phi\rangle$ 。

如果公式长度是输入长度的多项式：

$$|\langle\phi\rangle| \leq |w|^c$$

其中 c 是某个常数，那么构造这个公式也可以在多项式时间内完成，于是有：

$$A \leq_p SAT.$$

公式解释： $\leq_p$ 比 $\leq_m$ 多强调一个条件：归约函数必须能在多项式时间内计算。Cook-Levin 定理最终需要证明的是对任意 $A \in NP$ ，都有 $A \leq_p SAT$ 。

构造 ϕ_{cell} (P170)

ϕ 的变量: $x_{i,j,s}$, $i,j=1,\dots,n^k$, $s \in Q \cup \Gamma \cup \{\#\}$ //全体符号

$x_{i,j,s}$: 第 i 行第 j 列是否填了符号 s

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i,j \leq n^k} \{ [\bigvee_s x_{i,j,s}] \wedge [\bigwedge_{s \neq t} \overline{(x_{i,j,s} \wedge x_{i,j,t})}] \}$$

$$\bigvee_s x_{i,j,s} = 1 \Leftrightarrow (i,j)\text{格中至少有一个符号}$$

$$\bigwedge_{s \neq t} \overline{(x_{i,j,s} \wedge x_{i,j,t})} = 1 \Leftrightarrow (i,j)\text{格中至多有一个符号}$$

$$\text{例: } (x_{i,j,1} \vee x_{i,j,2} \vee x_{i,j,3}) \wedge \overline{(x_{i,j,1} \wedge x_{i,j,2})} \wedge \overline{(x_{i,j,1} \vee x_{i,j,3})} \wedge \overline{(x_{i,j,2} \vee x_{i,j,3})}$$

- 长 $O(n^{2k})$

- $\phi_{\text{cell}} = 1 \Leftrightarrow$ 每格有且只有一个符号;

构造 ϕ_{start} (P171)

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge \cdots \wedge x_{1,n^k,\#}$$

- 长 $O(n^k)$
- $\phi_{\text{start}} = 1 \Leftrightarrow$ 第一行是起始格局;

构造 ϕ_{accept} (P171)

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i, j, q_{\text{accept}}}$$

- 长 $O(n^{2k})$
- $\phi_{\text{accept}} = 1 \Leftrightarrow$ 表格中有接受状态

构造 ϕ_{move} (P171)

$$\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}.$$

ϕ_{move} 确定表的每行是上一行的合法结果.

只需判断每个 2×3 窗口是否“合法”.

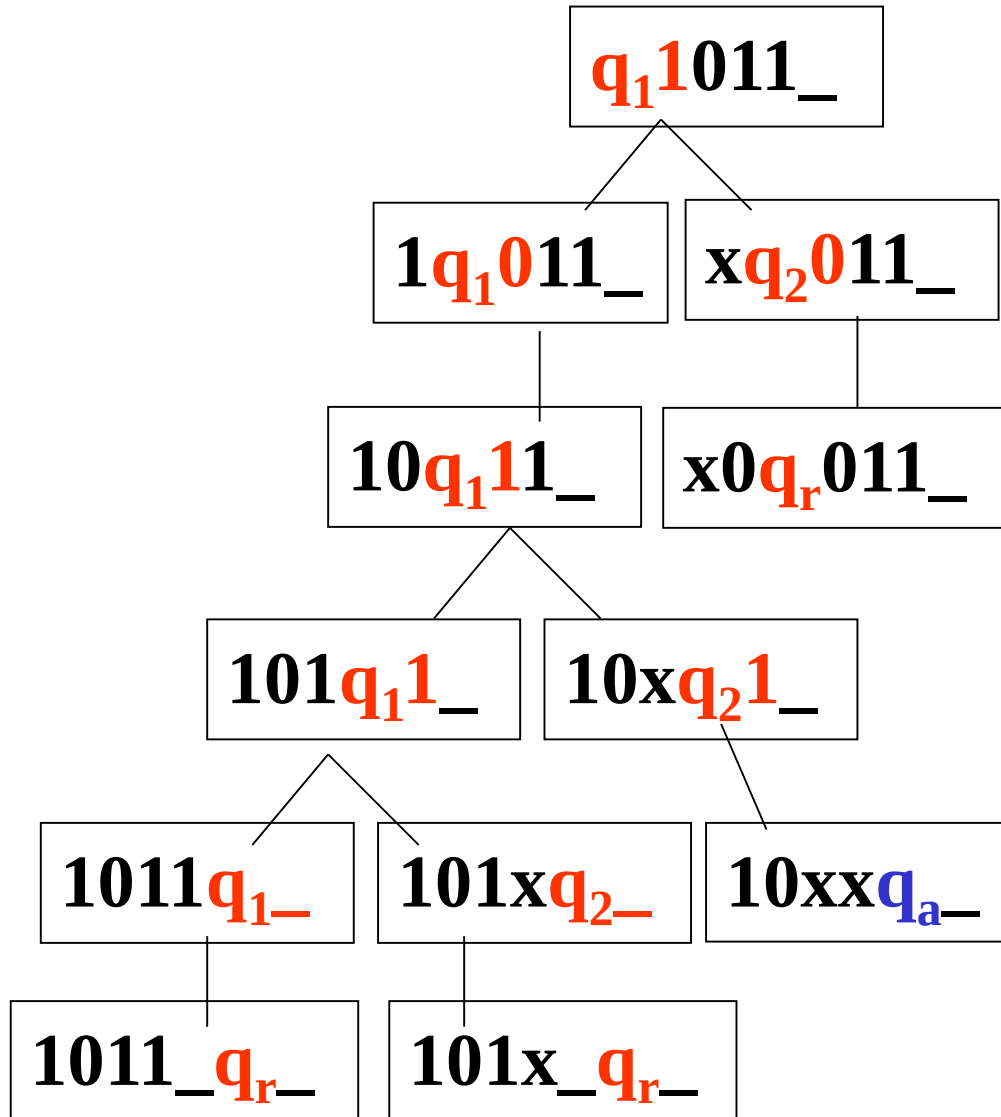
所谓 2×3 窗口, 是指相邻两行、连续三列构成的小块:

$$\begin{array}{ccc} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \end{array}$$

公式解释: 上面一行来自第 i 行, 下面一行来自第 $i+1$ 行. 三个连续列通常取为 $j-1, j, j+1$. 这个小窗口记录了某个局部区域在一步转移前后的变化.

图灵机一步转移只会影响读写头附近的局部位置: 状态可能移动一格, 被读符号可能被改写, 其他远处符号保持不变. 因此判断一个大表格相邻两行是否合法, 可以转化为检查所有局部窗口是否合法.

构造 ϕ_{move} (P171)



合法窗口(P171)

设 $\delta(q_1, a) = \{(q_1, b, R), \delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$,

合法
窗口

a	q ₁	b
q ₂	a	c

a	q ₁	b
a	a	q ₂

d	a	q ₁
d	a	b

#	b	a
#	b	a

a	b	a
a	b	q ₂

b	b	b
c	b	b

非法
窗口

a	b	a
a	a	a

a	q ₁	b
q ₁	a	a

a	q ₁	b
q ₁	a	q ₁

$$\phi_{\text{move}} = \bigwedge_{1 \leq i, j \leq n^k} \left\{ \bigvee_{\substack{a_1, a_2, \dots, a_6 \\ \text{是合法窗口}}} [x_{i, j-1, a_1} \wedge \dots \wedge x_{i+1, j+1, a_6}] \right\}$$

合法窗口

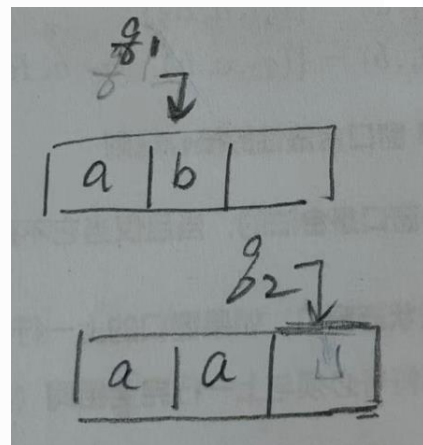
设 $\delta(q_1, a) = \{(q_1, b, R)\}$, $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$,

2. 窗口 2: 头读 a , 执行右移 (q_1, b, R)

表格

上	a	q_1	b
下	a	a	q_2

a	q_1	b
a	a	q_2



• 分析:

- 上一行中间列是状态 q_1 , 读入符号 a , 执行 $\delta(q_1, a) = (q_1, b, R)$: 头右移, 写入 b , 状态 q_1 应出现在下一行右侧列。
- 但这里下一行右侧列是 q_2 , 说明执行的是 $\delta(q_1, b) = (q_2, a, R)$: 头读入 b (上一行右侧列), 写入 a , 状态 q_2 出现在下一行右侧列。
- 结果: 上一行中间列 q_1 读入右侧的 b , 写入 a 到中间列, 状态 q_2 右移到下一行右侧列, 完全符合 $\delta(q_1, b) = (q_2, a, R)$ 。

• 结论: 合法。

合法窗口

设 $\delta(q_1, a) = \{(q_1, b, R), \delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$,

分析窗口矛盾

下行 ($t+1$ 时刻) 的格局, 无论从哪个分支看都不合法:

a	q_1	b
q_1	a	a

- **状态错误:** 根据两条规则, 转移后的状态都必须是 q_2 , 但下行的状态却是 q_1 。
- **读写头位置与字符错误:**
 - 若按分支 1 (左移), 读写头应移到左边, 中间格应是 c , 但下行中间是 a , 状态也不是 q_2 。
 - 若按分支 2 (右移), 读写头应移到右边, 中间格应是 a , 但状态应在右边格, 而不是左边格, 且状态也不是 q_2 。

结论: 非法窗口

合法窗口有常数个(P171)

设 $\delta(q_1, a) = \{(q_1, b, R), \delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$,

合法
窗口

a	q ₁	b
q ₂	a	c

#	b	a
#	b	a

a	q ₁	b
a	a	q ₂

a	b	a
a	b	q ₂

d	a	q ₁
d	a	b

b	b	b
c	b	b

$$\phi_{\text{move}} = \bigwedge_{1 \leq i, j \leq n^k} \left\{ \bigvee_{\substack{a_1, a_2, \dots, a_6 \\ \text{是合法窗口}}} [x_{i, j-1, a_1} \wedge \dots \wedge x_{i+1, j+1, a_6}] \right\} O(n^{2k})$$

N的一个转移函数规则对应常数个合法窗口
与N的转移函数无关的合法窗口有常数个

2×2窗口不能正确判断(补充)

设 $(q_2, c, L) \in \delta(q_1, b)$, $(q_3, f, L) \in \delta(q_1, e)$, a 是任意符号

合法
窗口

a	q₁	b
q₂	a	c

a	q₁	e
q₃	a	f

非法
窗口

a	q₁	b
q₃	a	c

$A \leq_p SAT$, SAT是NPC(P172)

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}$$

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \{ [\bigvee_s x_{i,j,s}] \wedge [\bigwedge_{s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}})] \}$$

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge \cdots \wedge x_{1,n^k,\#}$$

$$\phi_{\text{move}} = \bigwedge_{1 \leq i, j \leq n^k} \{ \bigvee_{\substack{a_1, a_2, \dots, a_6 \\ \text{是合法窗口}}} [x_{i,j-1,a_1} \wedge \cdots \wedge x_{i+1,j+1,a_6}] \}$$

$$(1) f(\mathbf{w}) = \langle \phi \rangle = \langle \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}} \rangle$$

$$(2) w \in A \Leftrightarrow \langle \phi \rangle \in SAT,$$

$$(3) \text{ 令 } |w| = \mathbf{n}, \text{ 则 } |\langle \phi \rangle| = \mathbf{O(n^{2k})}$$

$A \leq_p \text{SAT}$, SAT是NPC(P172)

- 可满足性问题SAT是NP中最核心的一类问题，只要能在多项式时间解决SAT，就能在多项式时间内解决所有NP问题。